FUNCIONES Y REGISTROS (STRUCT) EN C++

1. REGISTROS (STRUCTS) EN C++

Un Registro o “struct” es una estructura de datos heterogénea definida por el usuario. A diferencia de los arreglos (vectores o matrices) que solo pueden almacenar elementos del mismo tipo de dato, una estructura permite agrupar múltiples variables de DIFERENTES tipos de datos bajo un mismo nombre.

A cada variable individual dentro de un `struct` se le conoce como 'miembro' o 'campo'.

SINTAXIS BÁSICA:

```
struct NombreEstructura {
tipoDato1 miembro1;
tipoDato2 miembro2;
tipoDato3 miembro3;
};
```

EJEMPLO DE USO:

```
// Definición de la estructura para un Estudiante
struct Estudiante {
string nombre;
int edad;
float notaFinal;
};
// Declaración e inicialización de un registro
Estudiante e1;
e1.nombre = "Juan Pérez";
e1.edad = 20;
e1.notaFinal = 4.5;
```

VENTAJAS DE LOS REGISTROS:

- Permiten modelar entidades del mundo real (personas, vehículos, productos).
- Mantienen los datos organizados y cohesionados.
- Se pueden crear arreglos de estructuras (e.g., un vector de 50 estudiantes).

2. FUNCIONES EN C++

Una Función es un bloque de código reutilizable diseñado para realizar una tarea específica. La programación modular consiste en dividir un problema grande en subproblemas más pequeños resueltos mediante funciones.

ESTRUCTURA DE UNA FUNCIÓN:

```
tipoRetorno nombreFuncion(parametro1, parametro2, ...) {
// Cuerpo de la función
return valor; // Opcional según el tipoRetorno
}
```

TIPOS DE FUNCIONES:

a) Funciones sin retorno (void): Ejecutan una acción pero no devuelven ningún valor al programa principal.

b) Funciones con retorno: Devuelven un único valor del tipo especificado (int, float, bool, string, o incluso un struct).

PASO DE PARÁMETROS:

- Por Valor: La función recibe una copia del dato original. Modificar el parámetro dentro de la función NO afecta la variable original.
- Por Referencia (&): La función recibe acceso directo a la dirección de memoria de la variable. Modificar el parámetro SÍ altera el valor original.

3. INTEGRACIÓN: FUNCIONES + REGISTROS

Es una práctica habitual pasar estructuras a funciones para manipular o consultar sus datos de forma ordenada.

- Paso por referencia (&): Se usa para capturar o modificar datos del registro.

- Paso por referencia constante (const &): Se usa para leer/imprimir datos del registro de forma eficiente sin duplicar memoria ni permitir cambios.

EJEMPLO PRÁCTICO INTEGRADO:

```
#include <iostream>
#include <string>
using namespace std;
// 1. Declaración del registro
struct Estudiante {
string nombre;
int edad;
float notaFinal;
};
// 2. Prototipos de funciones
void ingresarDatos(Estudiante &e);
void mostrarDatos(const Estudiante &e);
bool estaAprobado(float nota);
int main() {
Estudiante est;
cout << "--- REGISTRO DE ESTUDIANTE ---" << endl;
ingresarDatos(est);
cout << "\n--- RESULTADO ---" << endl;
mostrarDatos(est);
return 0;
}
// Implementación de funciones
void ingresarDatos(Estudiante &e) {
cout << "Nombre: ";
```

```
getline(cin, e.nombre);

cout << "Edad: ";

cin >> e.edad;

cout << "Nota Final: ";

cin >> e.notaFinal;

}

bool estaAprobado(float nota) {

return nota >= 3.0;

}

void mostrarDatos(const Estudiante &e) {

cout << "Nombre: " << e.nombre << endl;

cout << "Edad: " << e.edad << " años" << endl;

cout << "Nota Final: " << e.notaFinal << endl;

cout << "Estado: " << (estaAprobado(e.notaFinal) ? "APROBADO" : "REPROBADO") <<
endl;
```